Complete CS Terminology Guide

Ordered by Learning Sequence - 26 Chapters

Math - Algorithms - OS - Networks - DB - Security - API

ML/AI - DL/LLM - DevOps - Linux - Testing - Career - IoT - ...

350+ Terms Every CS Student Must Know

Discrete Math | Programming | Algorithms | Mathematics | Theoretical CS | Hardware | OS | Networks | Databases
Security | API/Web | Cloud/DevOps | Data Science | Deep Learning | Data Eng. | Computer Vision | Compilers | Parallel Sys.
Soft. Eng. | Linux | Regex | Testing | Career | Ethics | IoT | Numerical

TABLE OF CONTENTS - 26 Chapters / 350+ Terms - Ordered by Learning Sequence

- CPU Scheduling
- Context Switching
- Mutex & Semaphore
- Deadlock
- Race Condition
- Virtual Memory & Paging
- Page Replacement
- Thrashing
- File System
- System Calls
- IPC Mechanisms
- Interrupts

8. COMPUTER NETWORKS

- OSI Model (7 Layers)
- TCP/IP Model
- MAC & ARP
- IP Addressing & Subnetting
- Routing
- TCP
- UDP
- DNS
- HTTP & HTTPS
- TLS/SSL
- Application Protocols
- NAT & DHCP
- CDN & Load Balancer
- VPN & Proxy

9. DATABASES

- Relational Model
- SQL DDL/DML/DCL
- SELECT & Filtering
- JOIN Types
- Normalization
- ER Diagram
- ACID
- Index
- Query Plan & EXPLAIN
- Transaction & Isolation
- Window Functions
- NoSQL Types & CAP
- Sharding & Replication
- ORM & Connection Pool

10. CYBERSECURITY

- CIA Triad
- Threat Modeling
- OWASP Top 10
- SQL Injection
- XSS
- CSRF
- Hashing & Salting
- Symmetric & Asymmetric
- PKI & Certificates
- Secret Management
- Penetration Testing
- Zero Trust
- DDoS & WAF

11. API & WEB DEVELOPMENT

- REST Principles
- HTTP Status Codes
- HTTP Headers
- Authentication vs AuthZ
- JWT
- OAuth 2.0 & OIDC
- CORS
- Rate Limiting
- GraphQL
- gRPC
- Caching Strategies
- Idempotency & Pagination
- Message Queue
- API Versioning & Docs

12. CLOUD COMPUTING & DevOps

- IaaS / PaaS / SaaS
- Core AWS Services
- IAM & Least Privilege
- Docker
- Docker Compose
- Kubernetes
- Infrastructure as Code
- CI/CD Pipeline
- Serverless & FaaS
- Observability
- Blue-Green & Canary
- GitOps
- Service Mesh

13. DATA SCIENCE & MACHINE LEARNING

- EDA
- Data Cleaning
- Feature Engineering
- Feature Selection
- Train / Val / Test Split
- Cross-Validation
- Bias-Variance Tradeoff
- Regularization
- Gradient Descent
- Evaluation Metrics
- Decision Tree & RF
- Gradient Boosting
- Clustering & Dim. Reduce.
- MLOps & Monitoring

14. DEEP LEARNING & LARGE LANGUAGE MODELS

- Perceptron & Layers
- Activation Functions
- Backpropagation
- Vanishing/Exploding Grad
- Batch Normalization
- CNN
- RNN / LSTM / GRU
- Transformer Architecture
- Self-Attention
- Transfer Learning & LoRA
- LLM Fundamentals
- Prompt Engineering
- RAG
- RLHF & Alignment
- Diffusion & Quantization

15. DATA ENGINEERING & BIG DATA

- ETL vs ELT
- Batch vs Streaming
- Apache Spark
- Apache Kafka
- Data Warehouse
- Data Lake & Lakehouse
- Columnar Format
- dbt
- Apache Airflow
- Data Quality
- Feature Store
- Data Catalog & Governance

16. COMPUTER VISION & SIGNAL PROCESSING

- Image Representation
- 2D Convolution
- Fourier Transform
- Image Filtering
- Edge Detection
- Morphological Operations
- Histogram & Thresholding
- Camera Model
- Object Detection
- Image Segmentation
- Optical Flow
- Data Augmentation

17. COMPILERS & LANGUAGE THEORY

- Lexical Analysis (Lexer)
- Parsing & Grammar
- AST
- Semantic Analysis
- Intermediate Representation
- Code Optimization
- Register Allocation
- Code Generation
- LLVM
- JIT Compilation
- Type System (Advanced)
- Garbage Collection (Deep)

18. PARALLEL & DISTRIBUTED SYSTEMS

- Amdahl & Gustafson
- Shared Memory Parallelism
- Message Passing (MPI)
- GPU Parallelism
- Distributed System Basics
- Vector Clock
- Raft Protocol
- Paxos
- Two-Phase Commit
- MapReduce
- Distributed Lock & CRDT
- Consistent Hashing

19. SOFTWARE ENGINEERING & SYSTEM DESIGN

- System Design Steps
- Scalability
- Caching Layers
- Event-Driven Architecture
- CQRS & Event Sourcing
- Circuit Breaker
- Twelve-Factor App
- Agile & Scrum
- Git Workflow
- Code Review Culture
- Technical Debt
- SRE & Blameless Culture

20. LINUX & BASH / TERMINAL

- Filesystem Hierarchy
- Basic Commands
- Permissions
- Pipe & Redirection
- Text Processing
- Process Management
- SSH & File Transfer
- Shell Scripting
- Environment Variables
- Crontab
- System Administration
- tmux & vim

21. REGEX & TEXT PROCESSING

- Basic Symbols
- Character Classes
- Quantifiers & Greedy/Lazy
- Groups & Backreferences

- Lookahead & Lookbehind
- grep & sed
- XML & HTML
- Log Analysis
- Python re Module
- JSON & CSV Parsing
- String Methods
- NLP Basics

22. TESTING & DEBUGGING TECHNIQUES

- Test Pyramid
- Mocking & Stubbing
- Integration Testing
- TDD
- Debugging Techniques
- Static Analysis
- Contract Testing
- Unit Testing
- Code Coverage
- E2E Testing
- Property-Based Testing
- Profiling
- Performance Testing

23. PROJECT MANAGEMENT & CAREER

- GitHub Essentials
- Git Workflow (Advanced)
- Technical CV
- System Design Interview
- Portfolio Projects
- Soft Skills
- Semantic Versioning
- Open Source Contribution
- LeetCode & DSA Prep
- Behavioral Interview
- Technical Writing
- Networking & Community

24. ETHICS & LAW (AI Ethics, GDPR, Licenses)

- GDPR Basics
- Data Minimization
- AI Ethics Framework
- Explainability (XAI)
- Software Licenses
- Liability & EU AI Act
- KVKK (Turkey)
- Privacy by Design
- Algorithmic Bias
- AI Safety
- Copyright & Patent
- Ethical Design

25. EMBEDDED SYSTEMS & IoT

- MCU vs MPU
- GPIO
- ADC & DAC
- RTOS
- Power Management
- MQTT
- Embedded Linux
- Arduino & ESP32
- PWM
- Communication Protocols
- Interrupt & Timer
- IoT Architecture
- OTA & Security

26. NUMERICAL METHODS & SCIENTIFIC COMPUTING

- IEEE 754 Floating Point
- Bisection Method
- Linear System Solving
- Numerical Integration
- Eigenvalue Computation
- FFT Applications
- Numerical Error Types
- Newton-Raphson
- Interpolation
- ODE Solvers
- Monte Carlo Methods
- Numerical Optimization

1. DISCRETE MATHEMATICS (The Mathematical Foundation of CS)

Where everything begins. The mathematical language of algorithm analysis, cryptography, and programming language theory. Logic first, then sets, combinatorics, and graphs.

Logic & Propositions
Proposition: a statement that is true or false. AND, OR, NOT, IMPLIES (->), IFF (<->). Tautology: always true. Contradiction: always false. CNF/DNF normal forms.

Set Theory
Set: unordered collection of unique elements. Union, intersection, difference, complement, power set. De Morgan: (A union B)' = A' inter B'. Cardinality |A|.

Proof Techniques
Direct proof: A->B directly. Contrapositive: !B->!A. Proof by contradiction: assume !B, derive contradiction. Mathematical induction: base + inductive step.

Mathematical Induction
P(1) true AND P(k)->P(k+1) => P(n) for all n. Strong induction: P(1)...P(k)->P(k+1). Structural induction: for recursive structures like trees and lists.

Modular Arithmetic
a mod n: remainder of dividing a by n. (a+b) mod n = ((a mod n)+(b mod n)) mod n. Fermat's little theorem: a^(p-1) mod p = 1. Foundation of RSA and hashing.

Combinatorics
Multiplication rule: n*m ways. Addition rule: n+m choices. Permutation P(n,r)=n!/(n-r)!. Combination C(n,r)=n!/(r!(n-r)!). Binomial theorem.

Pigeonhole Principle
If N+1 objects go into N boxes, at least one box has 2 objects. Generalization: ceil(N/k) objects in some box. Collision proofs, birthday attack analysis.

Inclusion-Exclusion
|A union B|=|A|+|B|-|A inter B|. Three sets: +|A|+|B|+|C|-|AB|-|AC|-|BC|+|ABC|. Used for derangements and surjection counting.

Recurrence Relations
T(n)=2T(n/2)+O(n) self-referential definition. Master Theorem: depending on a>=1, b>1, f(n) size gives O(n^log_b(a)) or O(n log n). Merge Sort example.

Graph Theory
Degree, path, cycle, connectivity. Euler path: each edge once. Hamiltonian path: each vertex once. Planar graph: Euler formula V-E+F=2. Bipartite graphs.

Tree Mathematics
A tree with N vertices has N-1 edges. Spanning tree. MST: Kruskal O(E log E), Prim O(E log V). Prufer sequence for tree encoding.

Probability & Expectation
Random variable X. E[X]=sum x*P(X=x). Var(X)=E[X2]-E[X]2. Linearity of expectation: E[X+Y]=E[X]+E[Y] without independence. Randomized algorithms.

Generating Functions
Sequence a0,a1,... -> G(x)=sum a_n*x^n. Closed-form derivation, recurrence solving, combinatorial identity proofs. Powerful algebraic tool.

2. PROGRAMMING FUNDAMENTALS & LANGUAGE CONCEPTS

First language concepts and memory model, then OOP, then paradigms and design patterns. Understanding how code works is the foundation before writing algorithms.

Compiled vs Interpreted
Compiled (C, Rust): source -> machine code, fast. Interpreted (Python, JS): line-by-line, flexible. JIT (JVM, V8): compile at runtime; best of both worlds.

Stack & Heap Memory
Stack: LIFO memory auto-allocated per function call (locals). Heap: dynamic allocation via malloc/new, manual or GC cleanup. Stack overflow, memory leak.

Pointer & Reference
Pointer: holds memory address of another variable (C/C++). Reference: alias. Dangling pointer: points to freed memory. Null pointer dereference -> segfault.

Type System
Static (C, Java, Rust): type checked at compile time. Dynamic (Python, JS): at runtime. Strong (Python): few implicit conversions. Weak (JS): coercions.

Scope & Lifetime
Scope: region where a variable is accessible. Lexical (static) scope: determined by code structure, not runtime. Closure: function captures its enclosing scope.

Garbage Collection
Mark-and-Sweep: trace reachability, collect unreachable. Reference Counting: Python. Generational GC: JVM young/old gen. Rust: ownership model, no GC.

OOP: 4 Pillars
Encapsulation (hide data), Abstraction (hide details), Inheritance (code reuse), Polymorphism (same interface, different behavior). Foundation of OOP.

SOLID Principles
Single Responsibility, Open/Closed (extend not modify), Liskov Substitution, Interface Segregation, Dependency Inversion. Maintainable design.

Exception Handling
try-catch-finally for error management. Checked (Java: compile-time enforced) vs Unchecked (runtime). raise/except (Python). Swallowing errors: anti-pattern.

Functional Programming
Pure functions: no side effects, same input same output. Immutability. Higher-order functions: take/return functions. Map, filter, reduce. Haskell, Scala.

Concurrency Primitives
Mutex: exclusive ownership lock. Semaphore: N-resource counter. Condition Variable: wait for condition. Atomic ops: indivisible. Python GIL, Java synchronized.

Generics & Templates
Type-independent reusable code. Java Generics <T>, C++ Template, Python Type Hints. Combines type safety with flexibility. Avoid code duplication.

Design Patterns
Creational: Singleton, Factory, Builder. Structural: Adapter, Decorator, Proxy. Behavioral: Observer, Strategy, Command, Iterator. GoF 23 patterns.

Metaprogramming
Code generation at compile time. C macros (text substitution). Rust proc-macro. Python decorator/metaclass. Reflection: inspect types at runtime.

3. ALGORITHMS & DATA STRUCTURES

First complexity analysis (why does it matter?), then fundamental structures (what?), then searching/sorting, then graph algorithms, then advanced techniques.

Big-O Notation
Upper bound on time/space complexity. O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n) linearithmic, O(n2) quadratic, O(2^n) exponential.

Amortized Analysis
Average cost per operation over N operations. Dynamic array: N pushes total O(N), each push amortized O(1). Accounting and potential methods.

Array vs Linked List
Array: O(1) index access, O(n) insert/delete, cache-friendly. Linked List: O(n) search, O(1) head/tail insert, extra pointer overhead. Array often faster.

Stack & Queue
Stack LIFO: push/pop O(1). Queue FIFO: enqueue/dequeue O(1). Deque: double-ended. Priority Queue: heap-backed O(log n) min/max. Call stack, BFS.

Hash Table
Hash function maps key to index. Chaining: linked list per bucket. Open Addressing: linear/quadratic/double probing. Load factor <0.7. Average O(1) CRUD.

Binary Search Tree
Left < root < right. Search/insert/delete O(h). Unbalanced BST degrades to O(n). AVL and Red-Black Tree guarantee O(log n). Inorder = sorted.

Heap (Min/Max)
Parent <= child (min-heap). Heapify O(n). Insert/extract O(log n). Array representation: parent=i, left=2i+1, right=2i+2. Heap Sort, Dijkstra, Prim.

Graph Representation
Adjacency Matrix: O(V2) space, O(1) access, dense graphs. Adjacency List: O(V+E) space, O(degree) access, sparse graphs. Edge List: simple.

Binary Search
O(log n) search in sorted array. Left, right, mid pointers. Off-by-one error is common. Applications: rotated array, first/last position, answer range.

BFS (Breadth-First)
Queue-based layer-by-layer traversal. Shortest path (unweighted). Level info. Bipartite check. O(V+E). Multi-source BFS for multiple starting points.

DFS (Depth-First)
Stack/recursion deep traversal. Topological sort (DAG). Cycle detection. Strongly connected components (Kosaraju/Tarjan). O(V+E).

Sorting Algorithms
Merge Sort: O(n log n) stable, O(n) extra. Quick Sort: O(n log n) avg, pivot matters. Tim Sort: hybrid, Python/Java default. Counting Sort: O(n+k).

Dijkstra's Algorithm
Single-source shortest path, no negative edges. Min-heap: O((V+E) log V). Relaxation: d[v]=min(d[v], d[u]+w(u,v)). Bellman-Ford supports negative.

Dynamic Programming
Overlapping subproblems + optimal substructure. Memoization (top-down, cache). Tabulation (bottom-up, table). Fibonacci, 0/1 Knapsack, LCS, Edit Distance.

Greedy Algorithms
Locally optimal choice at each step. Dijkstra, Huffman Coding, Activity Selection, Kruskal/Prim. Does not always guarantee global optimum; proof required.

Backtracking
Try all solutions, prune invalid branches. N-Queens, Sudoku, Permutations, Subset Sum. Exponential worst case but pruning makes it practical.

Approximation Algorithms
Guaranteed approximation ratio for NP-Hard problems in polynomial time. TSP: 2-approximation (metric). Vertex Cover: 2-approx. PTAS: (1+eps)-approx.

Complexity Classes
P: solvable in polynomial time. NP: verifiable in polynomial time. NP-Complete: in NP and NP-Hard. NP-Hard: at least as hard as NP. P=NP? Open.

4. MATHEMATICS FOR CS & AI

First linear algebra (the language of ML), then probability/statistics (handling uncertainty), then calculus/optimization (the learning mechanism). Order matters.

Vector & Matrix
Vector: n-dimensional point or direction. Matrix: MxN array. Matrix multiply (MxK)*(KxN)=(MxN). Transpose, determinant, trace. NumPy in practice.

Linear Independence
Vectors are linearly independent <=> none is a linear combination of others. Rank: number of independent rows/columns. Full rank: invertible matrix.

Eigenvalue & Eigenvector
Av=lambda*v. Eigenvalues reveal structural properties of a matrix. PCA: eigenvectors maximize variance. Spectral clustering, PageRank, graph analysis.

SVD
A=U*sigma*V^T. Low-rank matrix approximation. Dimensionality reduction. Recommender systems (matrix factorization). Generalization of PCA. scipy.linalg.svd.

Basic Probability
Sample space, event. P(A and B)=P(A)*P(B|A). Law of total probability. Bayes: P(A|B)=P(B|A)*P(A)/P(B). Independence: P(A and B)=P(A)*P(B).

Probability Distributions
Normal/Gaussian: mean+std, Central Limit Theorem. Bernoulli: single trial. Binomial: N trials. Poisson: events per time unit. Exponential: waiting time.

Statistical Estimation
MLE: find parameters that best explain the data. MAP: prior * likelihood. Bayesian inference: posterior distribution. Unbiased estimator, consistency.

Hypothesis Testing
H0 (null) vs H1 (alternative). p-value: probability of observation if H0 is true. alpha=0.05 significance. Type I (false reject), Type II (false accept).

Information Theory
Entropy H(X)=-sum P(x) log P(x): uncertainty measure. KL Divergence: distribution difference. Mutual Information. Cross-entropy as loss function.

Derivative & Gradient
df/dx single variable. Partial derivative: fix others. Gradient vector: direction of steepest ascent of a multivariate function. del f = (df/dx1,...,df/dxn).

Chain Rule
d/dx[f(g(x))]=f'(g(x))*g'(x). Sequential application for composite functions. The mathematical foundation of backpropagation in neural networks.

Convexity & Optimization
Convex function: local min = global min. Gradient descent converges to global optimum on convex. Lagrange multipliers: constrained optimization. KKT conditions.

Numerical Methods
IEEE 754 floating point: sign+exponent+mantissa. Machine epsilon ~2^-52. Newton-Raphson root finding. Numerical integration: Trapezoid, Simpson, Gaussian.

5. THEORETICAL CS & COMPUTABILITY

First the language hierarchy (Chomsky), then computation models, then complexity classes. Why are some problems unsolvable?

Chomsky Hierarchy
Type 3: Regular (DFA). Type 2: Context-Free (PDA). Type 1: Context-Sensitive (LBA). Type 0: Recursively Enumerable (TM). Each class contains lower ones.

Finite Automata (DFA/NFA)
DFA: single transition per state, deterministic. NFA: multiple transitions or epsilon. Every NFA can be converted to DFA (powerset construction). Recognizes regular languages.

Regular Languages & Pumping
Languages recognized by finite automata. Pumping Lemma: long strings must be pumpable. {a^n b^n} is not regular; proved by pumping lemma. Regex = regular language.

Context-Free Grammar
S -> aSb | epsilon production rules. Recognized by Pushdown Automata (PDA). CYK algorithm O(n3) parsing. Programming language grammars are CFG.

Turing Machine
Infinite tape, read/write head, state table. Universal TM simulates any TM. Church-Turing thesis: TM defines computability. Foundation of CS theory.

Halting Problem
No general algorithm can decide if a program halts. Proved by Cantor diagonal argument. Many problems reduce to halting; basis of undecidability proofs.

Rice's Theorem
Every nontrivial behavioral property of TM programs is undecidable. 'Does this program do X?' is undecidable in general. Limits of static analysis.

Class P
Decision problems solvable in deterministic polynomial time O(n^k). Sorting, searching, shortest path, MST are all in P. Efficiently solvable.

Class NP
Problems verifiable in polynomial time. Solvable by nondeterministic TM in polynomial. Every P problem is in NP. Certificate/witness concept.

NP-Complete
In NP AND every NP problem poly-reduces to it. Cook-Levin: SAT is first NP-C. 3-SAT, Vertex Cover, Clique, Hamiltonian Path, TSP are NP-Complete.

P vs NP
The Millennium Prize Problem. If P=NP: RSA breaks, protein folding is easy. General belief: P!=NP but unproven. $1M Clay Institute prize.

Reduction Technique
If A poly-reduces to B, solving B solves A. NP-hardness proof: reduce a known NP-C problem to the new problem. Shows relative difficulty.

6. COMPUTER HARDWARE & ARCHITECTURE

First the architectural model, then CPU internals, then memory hierarchy, then parallelism. Understanding why software behaves the way it does.

Von Neumann Architecture
CPU, memory, and I/O components. Fetch-Decode-Execute cycle. Von Neumann bottleneck: CPU-memory bandwidth limit. Foundation of modern computers.

CPU Components
ALU (arithmetic/logic), Control Unit, Registers (PC, SP, IR, general). Superscalar: multiple instructions per clock. Out-of-order execution for performance.

Instruction Pipeline
Fetch->Decode->Execute->Memory Access->Write Back. Pipeline hazards: Data (RAW/WAW/WAR), Control (branch prediction), Structural. Forwarding reduces stalls.

Branch Prediction
Predict if/else branches ahead of time to avoid pipeline stalls. Misprediction: flush + penalty cycles. BTB and PHT predictors. Spectre: side-channel exploit.

Cache Hierarchy
L1 (~4 cycle, ~32KB), L2 (~12 cycle, ~256KB), L3 (~40 cycle, ~8MB), RAM (~200 cycle), Disk (millions). Cache miss is extremely costly. Locality matters.

Cache Concepts
Cache line: 64 bytes. Hit ratio. MESI protocol (Modified/Exclusive/Shared/Invalid). False sharing: different threads write to same cache line. Performance killer.

Memory (DRAM & SRAM)
DRAM: dense, cheap, needs periodic refresh, used as RAM. SRAM: fast, expensive, no refresh, used as cache. Row/column addressing. Memory bandwidth bottleneck.

Endianness
Big-endian: high-order byte first (network). Little-endian: low-order byte first (x86). htons()/htonl() for conversion. Important in binary protocols.

SIMD
Single Instruction Multiple Data: process many data elements with one instruction. SSE, AVX (x86), NEON (ARM). Array ops, ML inference 4-8x speedup.

GPU Architecture
Thousands of small cores for massive parallelism. CUDA core, Warp (32 threads), Shared memory, Global memory. CPU: serial/powerful; GPU: parallel/numerous.

Virtual Memory
Page table maps virtual to physical addresses. TLB caches translations. Page fault: triggers disk read. /proc/PID/maps shows memory layout. ASLR security.

I/O & DMA
Polling: CPU repeatedly checks, wasteful. Interrupt-driven: hardware interrupts CPU, efficient. DMA: direct memory-device transfer bypassing CPU.

7. OPERATING SYSTEMS

First process/thread model, then scheduling, then synchronization, then memory management, then file systems. Foundation before advanced topics.

Process vs Thread
Process: independent address space, heavyweight. Thread: shared memory, lightweight. Process gives isolation; threads communicate easily. Context switch cost differs.

Process Lifecycle
New -> Ready -> Running -> Waiting -> Terminated. Preemption: OS forcibly switches running process. PCB (Process Control Block) stores process state.

CPU Scheduling
FCFS: arrival order. SJF: shortest job first, starvation risk. Round Robin: time quantum, fair. Priority scheduling. MLFQ: multi-level adaptive queues.

Context Switching
CPU saves registers, PC, SP and loads new process state. Has overhead: TLB flush, cache pollution. Excessive context switching degrades performance.

Mutex & Semaphore
Mutex: exclusive ownership, lock/unlock, has ownership. Binary Semaphore: similar but no ownership. Counting Semaphore: manages N resources.

Deadlock
Coffman conditions: Mutual Exclusion + Hold & Wait + No Preemption + Circular Wait. Prevention (break condition), Avoidance (Banker's), Detection/Recovery.

Race Condition
Two threads write same resource concurrently; result depends on order. Critical section protects shared state. Atomic operations (CAS) for lock-free alternatives.

Virtual Memory & Paging
Virtual->physical address via page table. TLB speeds up translation. Demand paging: load page when needed. Copy-on-Write: fork optimization.

Page Replacement
FIFO: evict oldest. LRU: evict least recently used. Optimal: look into future (impractical baseline). Clock: LRU approximation with circular buffer.

Thrashing
Excessive page faults: process spends more time paging than computing. Working set model as solution. Reduce multiprogramming degree.

File System
inode: metadata (size, permissions, timestamps, block pointers). Directory: name->inode mapping. Journaling: write-ahead log for consistency. VFS layer.

System Calls
User->kernel transition: mode switch. read, write, open, close, fork, exec, mmap, socket, ioctl are fundamental syscalls. strace for monitoring.

IPC Mechanisms
Pipe (one-way, related processes), Named Pipe/FIFO, Message Queue, Shared Memory (fastest), Signal (async notification), Unix Domain Socket.

Interrupts
Hardware IRQ (disk, NIC, timer) or software trap. ISR must be fast. Maskable vs non-maskable. Top half (quick, in interrupt context)/Bottom half (deferred).

8. COMPUTER NETWORKS

First the layered model (why?), then physical/link layer, then IP/routing, then TCP/UDP, then application protocols, finally security layer.

OSI Model (7 Layers)
Physical, Data Link, Network, Transport, Session, Presentation, Application. Each layer only talks to adjacent layers. Abstraction for troubleshooting.

TCP/IP Model
Practical 4-layer stack: Network Access (1-2), Internet (3), Transport (4), Application (5-7). The actual protocol suite powering the Internet.

MAC & ARP
MAC: 48-bit hardware address burned into NIC. ARP: resolves IP -> MAC via broadcast. ARP cache stores mappings. ARP spoofing: MITM attack vector.

IP Addressing & Subnetting
IPv4: 32-bit, 4 octets. IPv6: 128-bit, hex. CIDR /24 = 254 hosts. Subnet mask splits network/host. VLSM: variable-length subnets. NAT: private->public.

Routing
Router forwards packets at Layer 3 via IP. Static routes: manually configured. RIP: distance-vector, hop count. OSPF: link-state, Dijkstra. BGP: Internet backbone.

TCP
3-way handshake: SYN->SYN-ACK->ACK. Flow control: sliding window. Congestion control: slow start, congestion avoidance, fast retransmit. FIN/RST teardown.

UDP
Connectionless, fast, unreliable. Only 8-byte header. DNS, DHCP, NTP, streaming, gaming. Application layer adds reliability if needed (QUIC).

DNS
Recursive resolver->Root NS->TLD NS->Authoritative NS chain. Record types: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), TXT (verification).

HTTP & HTTPS
Request/Response: method, URL, version, headers, body. HTTP/1.1: persistent connections. HTTP/2: multiplexing, header compression. HTTP/3: QUIC (UDP-based).

TLS/SSL
ClientHello->ServerHello+Certificate->Key Exchange->Finished. Asymmetric for session key exchange, then symmetric AES encryption. Certificate chain.

Application Protocols
SSH: secure remote access, port 22. FTP/SFTP: file transfer. SMTP/IMAP/POP3: email. WebSocket: real-time. gRPC: HTTP/2 + Protobuf RPC framework.

NAT & DHCP
NAT: translates private IP to public via PAT with port numbers. DHCP DORA: Discover->Offer->Request->Ack. Lease time. DHCP starvation attack.

CDN & Load Balancer
CDN: distribute content to edge servers globally, reduce latency. LB: Round Robin, Least Connections, IP Hash. L4 (TCP) vs L7 (HTTP). HAProxy, Nginx.

VPN & Proxy
VPN: encrypted tunnel, IPSec/OpenVPN/WireGuard. Forward proxy: acts on behalf of client. Reverse proxy: sits in front of servers. MITM protection.

9. DATABASES

First relational model and SQL basics, then design (normalization, ER), then performance (indexes, query plans), then distributed (NoSQL, CAP).

Relational Model
Table (relation), row (tuple), column (attribute). Primary key: unique identifier. Foreign key: reference to another table. Referential integrity constraint.

SQL DDL/DML/DCL
DDL: CREATE/ALTER/DROP tables. DML: SELECT/INSERT/UPDATE/DELETE. DCL: GRANT/REVOKE. TCL: COMMIT/ROLLBACK/SAVEPOINT. Learn in this order.

SELECT & Filtering
SELECT cols FROM tbl WHERE cond ORDER BY col LIMIT n. Operators: =, !=, >, <, BETWEEN, IN, LIKE, IS NULL. DISTINCT for unique. GROUP BY + HAVING.

JOIN Types
INNER: matching rows only. LEFT: all of left + matching right. RIGHT: opposite. FULL OUTER: all of both. CROSS: cartesian product. SELF JOIN.

Normalization
1NF: atomic values, no repeating groups. 2NF: no partial dependencies. 3NF: no transitive dependencies. BCNF: every determinant is a candidate key.

ER Diagram
Entity (table), Attribute (column), Relationship. Cardinality: 1:1, 1:N, M:N. Weak entity, ISA/subtype hierarchy. Blueprint for physical schema.

ACID
Atomicity (all or nothing), Consistency (rules preserved), Isolation (concurrent transactions don't interfere), Durability (committed data persists).

Index
B-Tree: sorted search O(log n), range queries. Hash: equality O(1). Composite: multi-column. Partial: conditional. Covering: query served from index only.

Query Plan & EXPLAIN
EXPLAIN ANALYZE shows execution plan. Seq Scan vs Index Scan. Hash Join, Nested Loop, Merge Join. Optimizer uses statistics. Vacuum/Analyze to update.

Transaction & Isolation
Isolation levels: Read Uncommitted, Read Committed, Repeatable Read, Serializable. Anomalies: Dirty Read, Non-Repeatable Read, Phantom Read.

Window Functions
RANK/DENSE_RANK/ROW_NUMBER, LAG/LEAD, SUM/AVG OVER (PARTITION BY col ORDER BY col). Unlike GROUP BY: does not collapse rows. Powerful analytics.

NoSQL Types & CAP
Document (MongoDB), Key-Value (Redis), Column-Family (Cassandra), Graph (Neo4j). CAP: choose 2 of C+A+P. CP: MongoDB. AP: Cassandra. BASE: eventual consistency.

Sharding & Replication
Sharding: horizontal partitioning across servers. Range/Hash/Directory sharding. Replication: copies for redundancy. Master-Slave: read scaling. Raft/Paxos.

ORM & Connection Pool
SQLAlchemy, Hibernate, Prisma abstract SQL. N+1 problem: use DataLoader/eager load. PgBouncer/HikariCP: connection pooling reduces overhead.

10. CYBERSECURITY

First threat model, then web security (OWASP), then cryptography foundation, then network security, then advanced topics.

CIA Triad
Confidentiality, Integrity, Availability. The three pillars of security design. Evaluate every security control against these three properties.

Threat Modeling
STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege. Systematically identify assets, threats, and controls.

OWASP Top 10
Injection, Broken Auth, Sensitive Data Exposure, XXE, Broken Access Control, Security Misconfig, XSS, Insecure Deserialization, Vulnerable Components, Logging.

SQL Injection
User input bleeds into SQL query. ' OR 1=1 -- classic example. Prevent with prepared statements/parameterized queries. Blind, Time-based, Error-based types.

XSS
Stored (saved in DB), Reflected (from URL), DOM-based (JS manipulation). Prevent with CSP header, output encoding, DOMPurify. HttpOnly cookie.

CSRF
Forge requests using user's session. Prevent with SameSite=Strict cookie, CSRF token, Double-Submit Cookie. Use GET only for read operations.

Hashing & Salting
SHA-256: fast but weak for passwords. bcrypt/scrypt/Argon2: intentionally slow. Salt: per-user random value; defeats rainbow tables. Pepper: secret extra.

Symmetric & Asymmetric
AES-256 symmetric: same key, fast, bulk data. RSA/ECC asymmetric: public encrypts, private decrypts; for key exchange. TLS combines both.

PKI & Certificates
CA signs certificates. Trust Chain: Root CA->Intermediate->Leaf. SAN, wildcard certs. Certificate Pinning: MITM prevention. OCSP/CRL revocation.

Secret Management
Never hardcode API keys in source code. Use environment injection, not .env files. HashiCorp Vault, AWS Secrets Manager. Secrets stay in git history!

Penetration Testing
Recon->Scanning (Nmap)->Exploitation (Metasploit)->Post-Exploitation->Report. Black/White/Grey box. CVE numbering. CVSS severity scoring.

Zero Trust
Never trust, always verify. Micro-segmentation, least privilege, MFA, continuous verification. BeyondCorp by Google. Replacing perimeter security.

DDoS & WAF
Volumetric (UDP flood), Protocol (SYN flood), Application (HTTP flood). Rate limiting, Anycast, Cloudflare WAF, CAPTCHA mitigation strategies.

11. API & WEB DEVELOPMENT

First REST design principles, then authentication mechanisms, then advanced API patterns, then async messaging.

REST Principles
Stateless, Client-Server, Cacheable, Uniform Interface, Layered System. Resource URL: /users/42. HTTP methods: GET (read), POST (create), PUT (update), DELETE.

HTTP Status Codes
2xx: success (200 OK, 201 Created, 204 No Content). 3xx: redirect. 4xx: client error (400, 401, 403, 404, 409, 422). 5xx: server error (500, 502, 503).

HTTP Headers
Content-Type, Accept, Authorization: Bearer, Cache-Control, ETag, If-None-Match, X-Request-ID, X-Rate-Limit. CORS: Access-Control-Allow-Origin.

Authentication vs AuthZ
AuthN: who are you? AuthZ: what can you do? Order matters: verify identity first, then determine permissions. RBAC, ABAC, OAuth scopes.

JWT
Header.Payload.Signature. HS256 (shared secret) or RS256 (asymmetric). Access token short-lived (~15 min), refresh token long (~7 days). Revocation is hard.

OAuth 2.0 & OIDC
Authorization Code + PKCE: most secure for user apps. Client Credentials: service-to-service. Implicit: deprecated. OIDC adds id_token for identity.

CORS
Same-Origin Policy: browser security. Preflight OPTIONS checks permissions. Access-Control-Allow-Origin/Methods/Headers. Credentials: allow-credentials.

Rate Limiting
Token Bucket: allows bursts. Leaky Bucket: constant output. Fixed Window: simple but boundary attack. Sliding Window: fair. Redis for distributed RL.

GraphQL
Client queries exactly what it needs. No over/under-fetching. Query/Mutation/Subscription. N+1: solved by DataLoader. Introspection: self-documentation.

gRPC
HTTP/2 + Protocol Buffers. Unary, Server/Client/Bidirectional Streaming. .proto file defines service. 5-10x faster than JSON. Microservice communication.

Caching Strategies
Cache-Aside: app writes on miss. Write-Through: every write hits cache. Write-Behind: async. Read-Through: auto-populate on miss. Redis, Memcached.

Idempotency & Pagination
Idempotency: same request N times = 1 time result. Idempotency-Key header makes POST retry-safe. Cursor-based pagination: consistent and fast.

Message Queue
RabbitMQ: AMQP, exchange/binding/queue. Kafka: log-based, consumer groups, retention. SQS/SNS: managed. Async decoupling, retry, dead letter queue.

API Versioning & Docs
URL path /v1/users, header, or query param versioning. OpenAPI/Swagger: machine-readable API documentation. Backward compatibility is critical.

12. CLOUD COMPUTING & DevOps

First cloud service models, then core AWS services, then containers/orchestration, then CI/CD, then observability, then deployment strategies.

IaaS / PaaS / SaaS
IaaS: rent VMs, you manage everything (EC2). PaaS: deploy code, platform manages infra (Heroku). SaaS: use ready-made app (Gmail). Management decreases upward.

Core AWS Services
EC2 (VM), S3 (object storage), RDS (managed DB), Lambda (serverless), VPC (private network), IAM (identity), ECS/EKS (containers), CloudFront (CDN), Route53 (DNS).

IAM & Least Privilege
IAM: users, groups, roles, policies. Role assumption for service-to-service. Least privilege: only necessary permissions. MFA required. Condition keys for fine control.

Docker
Dockerfile: FROM, RUN, COPY, EXPOSE, CMD. Image: immutable layered template. Container: running instance. Layer cache speeds builds. Multi-stage: smaller images.

Docker Compose
Define multi-service setup in YAML. depends_on, networks, volumes, environment. Ideal for development. Use Kubernetes for production workloads.

Kubernetes
Pod (1+ containers), Deployment (replicas+rolling update), Service (stable IP/DNS), Ingress (external routing), ConfigMap/Secret, HPA (auto-scaling).

Infrastructure as Code
Terraform (HCL, multi-cloud): plan/apply/destroy. Pulumi (real languages). AWS CDK, CloudFormation. State file: current infra state. Drift detection.

CI/CD Pipeline
CI: auto build+lint+test on every push. CD Delivery: artifact ready for deployment. CD Deployment: auto-deploy. GitHub Actions, GitLab CI, Jenkins.

Serverless & FaaS
Event-driven invocation. Cold start latency. AWS Lambda, Google Cloud Functions, Azure Functions. Stateless; use Redis/DB for state storage.

Observability
Logs (ELK/Loki): event records. Metrics (Prometheus+Grafana): numeric measurements. Traces (Jaeger/Zipkin): request tracing. Alertmanager. SLA/SLO/SLI.

Blue-Green & Canary
Blue-Green: two identical environments, instant switch, instant rollback. Canary: 5%->20%->100% gradual rollout, monitor metrics. Feature flags for control.

GitOps
Git repo as single source of truth. ArgoCD/Flux keeps cluster in sync with repo. Drift detection and auto-remediation. Audit trail: every change in git.

Service Mesh
Sidecar proxy (Envoy) manages service-to-service communication. Auto mTLS, retry, circuit breaker, distributed tracing. Istio, Linkerd, Consul Connect.

13. DATA SCIENCE & MACHINE LEARNING

First data exploration (EDA), then data preparation, then model selection, then training/evaluation, then ensemble and advanced techniques.

EDA
Discover distributions, correlations, missing/outlier values. df.describe(), value_counts(), corr(). Histogram, boxplot, scatter matrix. Understanding data comes first.

Data Cleaning
Missing values: mean/median imputation, KNN imputation, drop rows/cols. Outliers: Z-score, IQR method, Isolation Forest. Type conversion, encoding.

Feature Engineering
Log/sqrt transform, one-hot encoding, target encoding, binning, polynomial features, interaction terms. The art of creating meaningful features from raw data.

Feature Selection
Filter (correlation, chi-square). Wrapper (RFE: recursive elimination). Embedded (Lasso L1, tree importance). Dimensionality reduction prevents overfitting.

Train / Val / Test Split
Train (70-80%): learning. Val (10-15%): hyperparameter selection. Test (10-15%): final evaluation. Never look at test set; if you do, it's invalid.

Cross-Validation
K-Fold: split into K folds, train+test K times. Stratified K-Fold: preserve class ratio. Nested CV: hyperparameter + model selection together.

Bias-Variance Tradeoff
Error = Bias2 + Variance + Noise. High bias: too simple (underfitting). High variance: too complex (overfitting). Increasing complexity: bias down, variance up.

Regularization
L1 Lasso: drives weights to zero, feature selection. L2 Ridge: keeps weights small, all features. ElasticNet: combines both. Dropout: for neural networks.

Gradient Descent
BGD: full data, stable. SGD: single sample, noisy. Mini-batch: balanced. Momentum: escape local minima. Adam: adaptive learning rate, most popular optimizer.

Evaluation Metrics
Classification: Accuracy, Precision=TP/(TP+FP), Recall=TP/(TP+FN), F1, AUC-ROC. Regression: RMSE, MAE, R2. Imbalanced data: prefer F1 and AUC.

Decision Tree & RF
DT: split by Gini/information gain; interpretable but high variance. RF: N trees + random features + bagging; OOB error for internal validation.

Gradient Boosting
Sequentially add trees on residual errors. XGBoost: L1/L2 reg, depth-first. LightGBM: leaf-wise, faster. CatBoost: native categorical feature support.

Clustering & Dim. Reduce.
K-Means: spherical clusters, predefined k. DBSCAN: noise-robust, shape-free. PCA: max variance, linear. UMAP: fast, preserves topology. t-SNE: visualization.

MLOps & Monitoring
Experiment tracking: MLflow, W&B. Model registry: version control. Serving: REST/gRPC. Data drift: Evidently. Model decay detection and retraining triggers.

14. DEEP LEARNING & LARGE LANGUAGE MODELS

First neural network basics, then training mechanics, then CNN, RNN, then Transformer, then LLMs and generative AI. Layered learning.

Perceptron & Layers
Single neuron: w*x+b then activation. Multi-layer: input->hidden->output. Universal approximation: sufficient hidden neurons approximate any function.

Activation Functions
Sigmoid: [0,1], gradient vanishing. Tanh: [-1,1], zero-centered. ReLU: max(0,x), fast, dying ReLU issue. Leaky ReLU: small negative slope. GELU: transformer standard.

Backpropagation
Apply chain rule to propagate loss gradient backward. dL/dw=dL/da*da/dz*dz/dw. Autograd (PyTorch/TF) handles this automatically. Every layer receives gradient.

Vanishing/Exploding Grad
In deep networks, gradients vanish (->0) or explode (->inf) going backward. Solutions: ReLU, Batch Norm, residual connections, gradient clipping.

Batch Normalization
Normalize activations over mini-batch: (x-mean)/std * gamma + beta. Stabilizes training, allows higher learning rates. Layer Norm preferred in transformers.

CNN
Conv: slide filter, produce feature map. Stride, padding, kernel size. Pooling: reduce dimensions. ResNet: skip connections for 100+ layers. EfficientNet, MobileNet.

RNN / LSTM / GRU
RNN: feedback loop for temporal memory; suffers gradient issues. LSTM: forget/input/output gates. GRU: 2 gates, simpler and faster. Seq2Seq models.

Transformer Architecture
Self-attention + Multi-head + FFN + Residual + Layer Norm. Positional encoding provides sequence order. Encoder-only (BERT), Decoder-only (GPT), Full (T5).

Self-Attention
Q, K, V: Attention(Q,K,V)=softmax(QK^T/sqrt(d_k))*V. Each token learns relationship weights with all others. Multi-head: different representation subspaces.

Transfer Learning & LoRA
Pre-trained model + fine-tuning: powerful model from limited data. Feature extraction: freeze base, train new head. LoRA: low-rank matrices, ~1% of params.

LLM Fundamentals
Pre-training: next-token prediction, billions of tokens. Scaling laws: larger model = better performance. Emergent abilities: capabilities not explicitly trained.

Prompt Engineering
Zero-shot, few-shot (show examples), chain-of-thought (step-by-step), ReAct (reason+act). System/user/assistant roles. Temperature, top-p sampling.

RAG
Question -> embedding -> vector DB search -> relevant docs + question -> LLM -> answer. Reduces hallucination. Chunking, re-ranking, hybrid search matter.

RLHF & Alignment
SFT -> Reward Model training -> PPO policy optimization. Preference data: human comparison pairs. DPO: reward-model-free alternative. ChatGPT's secret sauce.

Diffusion & Quantization
Diffusion: add noise (forward) + remove (reverse). Stable Diffusion, FLUX. Quantization: float32->int8 model compression. GPTQ, GGUF for local LLM.

15. DATA ENGINEERING & BIG DATA

First pipeline and ETL concepts, then processing engines (Spark/Kafka), then storage (warehouse/lake), then orchestration, then quality and governance.

ETL vs ELT
ETL: extract->transform->load (traditional warehouse). ELT: extract->load->transform (cloud warehouse: Snowflake, BigQuery). ELT is more flexible and scalable.

Batch vs Streaming
Batch: periodic large processing (nightly ETL). Streaming: continuous small processing (ms-seconds). Lambda: both in parallel. Kappa: streaming only. Flink.

Apache Spark
In-memory distributed computation. RDD (low-level), DataFrame (optimized), Dataset (typed). Lazy evaluation: waits for action before executing transformations.

Apache Kafka
Log-based distributed messaging. Topic->Partition (ordering guarantee)->Offset. Consumer Group: parallel reading. Retention: data kept for N days.

Data Warehouse
OLAP: read-heavy, write-light, columnar. Star schema: fact + dimension tables. SCD (Slowly Changing Dimension) types. Snowflake, BigQuery, Redshift.

Data Lake & Lakehouse
Data Lake: raw data, any format, cheap (S3/GCS). Lakehouse: lake + warehouse features. Delta Lake, Apache Iceberg, Hudi: ACID + versioning on lake.

Columnar Format
Parquet, ORC: columnar storage, high compression, predicate pushdown. 10-100x faster analytics vs CSV/JSON. Vectorization support for fast processing.

dbt
SQL-based data transformation. Model (SELECT as table/view). ref() builds dependency graph. Tests, documentation, lineage. Center of Analytics Engineering.

Apache Airflow
Python DAG for workflow scheduling. Operator (task type), Sensor (wait), Hook (connection). Backfill, retry, SLA miss alerts, XCom for data passing.

Data Quality
Completeness, Accuracy, Consistency, Timeliness, Uniqueness. Great Expectations, Soda for automated checks. Data contract: producer's responsibility.

Feature Store
Online (low latency, serving) vs Offline (batch, training). Prevents training-serving skew. Reusable features across teams. Feast, Tecton, Hopsworks.

Data Catalog & Governance
Centralized metadata, lineage, data dictionary. RBAC access control. GDPR compliance. Data masking, anonymization, audit logging.

16. COMPUTER VISION & SIGNAL PROCESSING

First image representation, then mathematical operations (convolution, Fourier), then classical CV techniques, then deep learning-based CV.

Image Representation
Grayscale: 2D matrix [H x W]. Color: 3D tensor [H x W x C]. RGB, BGR (OpenCV default), HSV (color selection), LAB (perceptual distance). uint8 [0-255].

2D Convolution
Output pixel = filter dot product with image patch. Stride: step size. Padding: zero/reflect. Separable conv: computation savings. Depthwise convolution.

Fourier Transform
Signal -> frequency domain. FFT: O(n log n). Low freq: global structure. High freq: edges/noise. Convolution theorem: spatial conv = frequency multiply.

Image Filtering
Gaussian blur: noise reduction. Median: salt-pepper noise. Sharpening: boost high freq. Sobel/Laplacian: edge detection. Bilateral: blur preserving edges.

Edge Detection
Sobel: x+y gradient magnitude. Canny: Gaussian->Sobel->Non-Max Suppression->Hysteresis thresholding. LoG: Laplacian of Gaussian. Threshold matters.

Morphological Operations
Erosion: shrink objects. Dilation: grow objects. Opening=Erosion+Dilation: remove noise. Closing=Dilation+Erosion: fill holes. Structuring element shape.

Histogram & Thresholding
Histogram: pixel intensity distribution. Histogram equalization: enhance contrast. Otsu: automatic optimal threshold. Adaptive threshold: local region-based.

Camera Model
Pinhole: 3D->2D projection. Intrinsics: focal length, principal point. Distortion: radial/tangential. OpenCV calibrateCamera(). Stereo: depth estimation.

Object Detection
HOG+SVM (classical). R-CNN family: region proposal. YOLO: single-pass full image, real-time. SSD: multi-scale. Anchor box, IoU, NMS concepts.

Image Segmentation
Semantic: class label per pixel. Instance: separate each object instance. Panoptic: both combined. U-Net (medical), Mask R-CNN, SAM (foundation model).

Optical Flow
Pixel motion vectors between consecutive frames. Lucas-Kanade: sparse, corner points. Farneback: dense, all pixels. RAFT: deep learning based.

Data Augmentation
Flip, rotate, crop, color jitter, mixup, cutmix, mosaic. Artificially enlarges training set. Albumentations, torchvision.transforms. Reduces overfitting.

17. COMPILERS & LANGUAGE THEORY

First lexical analysis, then parsing, then semantic analysis, then optimization, then code generation. The front-to-back flow of a compiler.

Lexical Analysis (Lexer)
Tokenize source code. Token types: keyword, identifier, literal, operator, punctuation. Defined by regex. flex/lex tools for auto-generation. Runs as DFA.

Parsing & Grammar
Tokens -> Parse Tree. Top-down: recursive descent, LL(k). Bottom-up: LR, LALR(1) (yacc/bison). Ambiguous grammar: multiple parse trees. Precedence rules.

AST
Parse tree stripped of redundant nodes. Foundation for compilers, interpreters, linters, formatters. Visitor pattern for traversal. Key intermediate artifact.

Semantic Analysis
Type checking, scope resolution, type inference. Symbol table: variable/function metadata. Undeclared variable, type mismatch, return type errors.

Intermediate Representation
Source-language-independent, machine-near format. SSA: each variable assigned once. LLVM IR: target-independent. Three-address code: a = b op c.

Code Optimization
Constant folding/propagation, dead code elimination, loop unrolling/invariant, function inlining, CSE. Optimization levels: -O0/-O1/-O2/-O3.

Register Allocation
Unlimited virtual registers -> limited physical. Graph coloring on interference graph. Spill: write to memory when no register available. Live variable analysis.

Code Generation
IR -> target machine code. Instruction selection: pattern matching. x86, ARM, RISC-V targets. Calling convention: argument/return value passing.

LLVM
Modular compiler infrastructure. Clang, Rust, Swift use LLVM. Pass system for optimization. JIT compilation support. MLIR: multi-level IR for ML compilers.

JIT Compilation
Compile bytecode/IR to native code at runtime. JVM HotSpot, V8, PyPy. Profile-guided: aggressively optimize hot paths. AOT vs JIT tradeoff.

Type System (Advanced)
Hindley-Milner type inference: ML, Haskell, Rust. Dependent types. Gradual typing: Python mypy. Subtyping and variance (covariance/contravariance).

Garbage Collection (Deep)
Mark-Sweep: trace reachability graph. Generational: young gen (frequent) old gen (rare). Incremental: reduce pause times. Rust: ownership+borrow, no GC.

18. PARALLEL & DISTRIBUTED SYSTEMS

First parallel computing laws, then shared/distributed memory models, then distributed system fundamentals, then consensus, then advanced topics.

Amdahl & Gustafson

Amdahl: Speedup=1/(S+(1-S)/N), serial fraction sets ceiling. Gustafson: problem grows with more processors; more optimistic for scalable systems.

Shared Memory Parallelism

OpenMP pragmas: threads share memory. parallel, critical, atomic, barrier, reduction directives. Race condition prevented by mutex/atomic ops.

Message Passing (MPI)

MPI_Send/Recv for distributed memory systems. Broadcast, Scatter, Gather, Reduce collective ops. Deadlock risk: plan communication order carefully.

GPU Parallelism

CUDA: Grid->Block->Thread hierarchy. __global__ kernel. cudaMalloc/cudaMemcpy. Shared memory fast within warp. Warp divergence kills performance.

Distributed System Basics

Partial failure, asynchrony, no global clock. Network partitions can happen any time. Byzantine fault: malicious node sends wrong info. CAP tradeoffs.

Vector Clock

Each node tracks its own counter. Happens-before: VC(A) < VC(B) means A precedes B. Causality tracking. Generalization of Lamport clock. Conflict detection.

Raft Protocol

Leader election: highest term + log wins. Log replication: leader writes, majority commits. Term: monotonically increasing number. Prevents split-brain.

Paxos

Prepare/Promise/Accept/Accepted phases. Multi-Paxos for log replication. Hard to understand; Raft is more pedagogical alternative. ZooKeeper uses ZAB.

Two-Phase Commit

Coordinator: prepare->commit/abort. All ready=commit, one no=abort. Coordinator failure causes blocking. 3PC reduces blocking but adds complexity.

MapReduce

Map: (k1,v1)->list(k2,v2). Shuffle: group same k2. Reduce: (k2,list(v2))->list(v3). Foundation of Hadoop. Spark is more flexible in-memory alternative.

Distributed Lock & CRDT

Distributed lock: Redis Redlock, ZooKeeper. TTL prevents deadlock. CRDT: automatic conflict-free merge without consensus. Counter, Set, OR-Set types.

Consistent Hashing

Adding/removing servers moves only K/N keys. Virtual nodes for load balance. Ring topology. Standard in distributed caches and DHT systems.

19. SOFTWARE ENGINEERING & SYSTEM DESIGN

First design principles, then architectural decisions, then large-scale system design, then software process and culture.

System Design Steps
Clarify requirements (functional/non-functional) -> Capacity estimation (QPS, storage) -> High-level architecture -> Data model -> API -> Bottleneck & scale.

Scalability
Vertical: more powerful machine (has limits). Horizontal: more machines (elastic). Stateless architecture enables horizontal scaling. Auto-scaling.

Caching Layers
Browser cache->CDN->Reverse proxy->App cache (Redis)->DB cache. Each layer reduces latency and load. Cache invalidation is notoriously hard.

Event-Driven Architecture
Producer->Broker (Kafka)->Consumer. Async, loose coupling. Saga pattern: distributed transaction management. Outbox pattern: prevent event loss.

CQRS & Event Sourcing
CQRS: separate read and write models; easier scaling. Event Sourcing: store events not state, replay anytime. Complete audit trail for free.

Circuit Breaker
Open circuit when downstream fails to prevent cascade. Half-open: retry after timeout. Resilience4j, Hystrix. Prevents cascading failure across services.

Twelve-Factor App
Codebase, Deps, Config (env), Backing Services, Build/Release/Run, Stateless Processes, Port Binding, Concurrency, Disposability, Dev/Prod parity, Logs, Admin.

Agile & Scrum
Sprint (2 weeks), Daily Standup, Sprint Review, Retrospective, Product/Sprint Backlog. Velocity measures capacity. Kanban: flow-focused alternative.

Git Workflow
Feature branch->PR->Code Review->Squash merge->main. Trunk-Based: short-lived branches, feature flags. Conventional Commits: semantic messages.

Code Review Culture
Small, focused PRs. Descriptive description. LGTM, nit, blocker labels. Check code, tests, security, performance. Constructive feedback; no ego.

Technical Debt
Intentional (quick fix) vs unintentional (ignorance). Refactoring: change structure, preserve behavior. SonarQube for measurement. Debt accrues interest.

SRE & Blameless Culture
Error Budget: 1-SLO; depleted means feature freeze. Toil reduction. Blameless post-mortem: blame the system, not people. Runbook, chaos engineering.

20. LINUX & BASH / TERMINAL

First filesystem and basic commands, then permissions and process management, then text processing tools, then scripting, then system administration.

Filesystem Hierarchy
/: root. /bin: essential binaries. /etc: configuration. /home: user dirs. /var: variable data (logs). /tmp: temporary. /proc: kernel virtual FS. /dev: devices.

Basic Commands
ls -la, cd, pwd, mkdir -p, rm -rf, cp -r, mv, touch, cat, less, head/tail -f. find . -name '*.py' -type f. tree for directory visualization.

Permissions
rwxr-xr-- = 754. chmod 755, chown user:group. setuid/setgid: run as another user. sticky bit: /tmp protection. umask sets default permissions.

Pipe & Redirection
| pipes stdout to next command. > overwrite, >> append. 2> redirect stderr, 2>&1 combine. /dev/null discard. tee: write to both screen and file.

Text Processing
grep -rn pattern for file search. sed 's/old/new/g' in-place replace. awk '{print $2}' field processing. sort | uniq -c | sort -rn for frequency.

Process Management
ps aux, top/htop. kill -9 PID. & background. jobs, fg, bg. nohup: survive terminal close. Ctrl+C (SIGINT), Ctrl+Z (SIGTSTP) signals.

SSH & File Transfer
ssh user@host -p port -i key.pem. ssh-keygen for RSA/Ed25519 keypair. authorized_keys on server. scp for copying. rsync -avz for incremental sync.

Shell Scripting
#!/bin/bash shebang. Variable: NAME='ali'. $1 $2 positional args. $? exit code. if/elif/else, for/while loops. set -e: exit on error. set -x: debug.

Environment Variables
export VAR=value. .bashrc/.zshrc for persistence. env to list. $PATH: command search path. $HOME, $USER, $SHELL. source to reload config.

Crontab
crontab -e to edit. '*/5 * * * * /script.sh' every 5 min. 5 fields: min hour day month weekday. @reboot on startup. @daily shorthand.

System Administration
df -h disk, du -sh* folder size, free -h RAM, uname -a kernel. ss -tulpn open ports. journalctl -f system logs. systemctl start/stop/status service.

tmux & vim
tmux: terminal multiplexer; Ctrl-b prefix, new/attach/detach sessions. vim: i=insert, Esc=normal, :w save, :q quit, dd delete line, /search.

21. REGEX & TEXT PROCESSING

First basic symbols, then character classes and quantifiers, then groups and lookahead, then language/tool-specific usage, then NLP basics.

Basic Symbols
. matches any char. * zero or more. + one or more. ? zero or one. ^ line start, $ line end. | or. \ escape. Mastering these makes you powerful.

Character Classes
[a-z] lowercase. [A-Z] uppercase. [0-9] digit. \d digit, \w word char, \s whitespace. \D/\W/\S negations. [^abc] not these characters.

Quantifiers & Greedy/Lazy
a{3} exactly 3. a{2,5} between 2-5. .* greedy (match most). .*? lazy (match least). Greedy backtracking impacts performance. Possessive: no backtrack.

Groups & Backreferences
(abc) capturing group. (?:abc) non-capturing. \1 backreference. (?P<name>...) named group. Access with re.group(1). Capture groups essential for parsing.

Lookahead & Lookbehind
(?=abc) positive lookahead. (?!abc) negative. (?<=abc) positive lookbehind. (?<!abc) negative. Non-consuming: match context without including it.

Python re Module
re.match (from start), re.search (anywhere), re.findall (list), re.finditer (iterator), re.sub (replace), re.compile (pre-compile). Flags: IGNORECASE, MULTILINE.

grep & sed
grep -E extended regex. grep -P Perl-compatible. grep -rn all files. sed 's/pat/rep/g' global. sed -i in-place. sed -n '/start/,/end/p' range print.

JSON & CSV Parsing
json.loads/dumps. jq '.data[].name' CLI JSON processing. csv.DictReader. pandas read_csv(). Watch encoding/delimiter/quoting. Schema validation.

XML & HTML
xml.etree.ElementTree standard lib. lxml fast. BeautifulSoup: find(), find_all(), CSS selector. XPath for advanced access. Never parse HTML with regex.

String Methods
split/strip/join/replace/startswith/endswith. f-string: f'{var:.2f}'. encode/decode. zfill/ljust/rjust. Python str is immutable; every op creates new object.

Log Analysis
access.log: IP date method URL status size. grep | awk | sort | uniq -c | head pipeline. GoAccess for visual. ELK Stack: Elasticsearch+Logstash+Kibana.

NLP Basics
Tokenization, lowercasing, stop word removal, stemming (Porter), lemmatization (WordNet/spaCy). TF-IDF, bag-of-words. NLTK and spaCy Python libraries.

22. TESTING & DEBUGGING TECHNIQUES

First testing philosophy and pyramid, then unit test writing, then integration/E2E, then TDD, then debugging and profiling techniques.

Test Pyramid
Unit (many, fast, cheap)->Integration (medium)->E2E (few, slow, expensive). Ice cream cone anti-pattern: over-relying on E2E. Balance is critical.

Unit Testing
Test single function/class in isolation. pytest, JUnit, Jest. AAA: Arrange-Act-Assert. Mock external dependencies. Each test independent; order shouldn't matter.

Mocking & Stubbing
Mock: fake implementation, verifies calls. Stub: returns fixed value. Spy: real implementation + call tracking. unittest.mock, Mockito, Jest mock.

Code Coverage
Line, Branch, Function, Condition coverage. coverage.py, Istanbul/nyc. 80%+ target but doesn't guarantee quality. Mutation testing is more powerful.

Integration Testing
Test multiple components together. Real DB, API, messaging. Testcontainers: isolated Docker environment. Slower but more realistic than unit tests.

E2E Testing
Full user scenario testing. Playwright, Cypress, Selenium. Flaky tests from async timing. Parallel execution, retry logic, screenshot on failure.

TDD
Red: write failing test. Green: minimum code to pass. Refactor: clean up. Forces thinking before coding. Improves design quality. Kent Beck popularized.

Property-Based Testing
Generate random inputs and verify properties hold. Hypothesis (Python), QuickCheck (Haskell). Auto-discovers edge cases. Complements classic unit tests.

Debugging Techniques
Print/log (simple). IDE breakpoint + watch (powerful). Binary search debugging: halve the problem. Rubber duck: explain to someone. Git bisect: find regression.

Profiling
CPU: cProfile (Python), async-profiler (Java). Memory: tracemalloc, Memray, Valgrind. Flame graph: visual hotspot analysis. Find bottleneck before optimizing.

Static Analysis
Detect bugs and style issues without running code. pylint, flake8, mypy, ruff (Python). ESLint (JS). SonarQube (multi-language). Pre-commit hook integration.

Performance Testing
Load: normal load. Stress: beyond limits. Soak: sustained medium load. Spike: sudden surge. k6, Locust, JMeter, Gatling. Percentile metrics (p95, p99).

Contract Testing
Both sides of microservice API test the contract. Pact framework. Consumer-Driven Contract: consumer defines expectations. Mandatory in CI pipeline.

23. PROJECT MANAGEMENT & CAREER

First GitHub and workflow, then career preparation, then interview techniques, then long-term career and community.

GitHub Essentials
README.md, .gitignore, LICENSE, CONTRIBUTING.md. Issues: bugs/tasks. PR: code review. GitHub Actions: CI/CD. Projects: Kanban board. Releases: versioning.

Semantic Versioning
MAJOR.MINOR.PATCH. Major: breaking change. Minor: backward-compatible new feature. Patch: bug fix. 0.x.y: development phase. CHANGELOG.md for history.

Git Workflow (Advanced)
git stash for temporary save. cherry-pick: take specific commit. rebase -i: clean up commit history. reflog: recover lost commits. bisect: find regression.

Open Source Contribution
Start with 'good first issue'. Read CONTRIBUTING.md. Fork->branch->commit->PR. CLA signing. Opening issues is also contributing. Watching teaches you.

Technical CV
Concrete metrics: '30% latency reduction', '10K users'. List tech stack with context. GitHub link, personal site. 1 page. ATS: include relevant keywords.

LeetCode & DSA Prep
Easy->Medium->Hard progression. Blind 75 / NeetCode 150 list. Patterns: sliding window, two pointer, BFS/DFS, DP. Always study solution after each problem.

System Design Interview
Ask clarifying questions first. Design top-down. Estimate QPS/storage. Explain trade-offs. Defend CAP, sharding, caching, SQL vs NoSQL decisions.

Behavioral Interview
STAR method: Situation, Task, Action, Result. Prepare conflict, failure, leadership, collaboration stories. Research company values. Ask questions.

Portfolio Projects
Full-stack app, CLI tool, ML model, API service, open source contribution. Deploy it (Vercel, Railway). README: problem, solution, tech, demo link, impact.

Technical Writing
Docstring: function purpose, parameters, return. OpenAPI/Swagger: machine-readable API docs. ADR (Architecture Decision Record). RFC writing skills.

Soft Skills
Written communication: critical in async remote work. Explain technical concepts to non-technical. Time management: timeboxing, Pomodoro. Saying no is a skill.

Networking & Community
Active LinkedIn. Meetup/conferences. CS communities on Twitter/X. Discord: Reactiflux, ML servers. Writing a blog builds visibility. Teaching reinforces learning.

24. ETHICS & LAW (AI Ethics, GDPR, Licenses)

First data privacy regulations, then AI ethics fundamentals, then algorithmic issues, then legal framework and responsibility.

GDPR Basics
EU data protection regulation. Lawful basis for personal data processing (consent, legitimate interest). Rights: access, rectification, erasure (right to be forgotten).

KVKK (Turkey)
Turkish personal data protection law. Data controller + data processor. Obligation to inform. Explicit consent. KVKK violation: administrative fine. Similar to GDPR.

Data Minimization
Collect only what's necessary for the purpose. Purpose limitation: can't use data for other purposes. Storage limitation: delete when no longer needed.

Privacy by Design
Build privacy in from the start, not as an afterthought. GDPR Article 25. Default settings should be most private. Minimize data flow from the beginning.

AI Ethics Framework
Fairness, Accountability, Transparency, Ethics (FATE). EU AI Act and IEEE Ethically Aligned Design frameworks. Building trustworthy AI systems.

Algorithmic Bias
Historical discrimination in training data reflects in model. Disparate impact: different outcomes for different groups. Fairness metrics: demographic parity, EO.

Explainability (XAI)
SHAP: each feature's contribution to model output. LIME: local approximation. Grad-CAM: CNN decision region. GDPR's 'right to explanation' challenges black-box.

AI Safety
Alignment problem: is AI aligned with human values? Corrigibility: can we correct it? Interpretability: can we understand it? Adversarial attack robustness.

Software Licenses
MIT: permissive, attribution required. Apache 2.0: patent protection included. GPL: derivative must also be GPL (copyleft). LGPL: library use free. Proprietary.

Copyright & Patent
Code automatically gets copyright. Patent: for inventions, limited in software. Trade Secret: protected by NDA. AI-generated content copyright is debated.

Liability & EU AI Act
Who's responsible for AI-caused harm? Developer, operator, user. Risk categories: unacceptable (mass facial recognition), high (credit scoring), limited, minimal.

Ethical Design
Value Sensitive Design: integrate stakeholder values into design process. Inclusive design: consider diverse users. Dark patterns: manipulative UI to avoid.

25. EMBEDDED SYSTEMS & IoT

First hardware fundamentals (MCU/MPU), then peripheral interfaces, then RTOS, then IoT communication protocols, then security and OTA.

MCU vs MPU
MCU: CPU+RAM+Flash+peripherals on one chip (Arduino, STM32, ESP32), low power/cost. MPU: powerful CPU, needs external RAM (Raspberry Pi). Scenario determines choice.

Arduino & ESP32
Arduino: AVR/ARM, C++, simple ecosystem. ESP32: WiFi+Bluetooth built-in, FreeRTOS support, dual-core. Both support GPIO, PWM, ADC, I2C, SPI, UART.

GPIO
Digital input (button reading) and output (LED control). Pull-up/pull-down resistors. Active HIGH/LOW logic. Debouncing: filter mechanical button bounce in software.

PWM
Analog effect with digital signal. Duty cycle: ON time / period ratio. LED brightness, servo angle, DC motor speed. Generated by Timer/Counter hardware.

ADC & DAC
ADC: analog (temperature, light)->digital. 10-bit=1024 steps, 12-bit=4096. DAC: digital->analog. Audio output, reference voltage. Nyquist: sample >= 2*fmax.

Communication Protocols
I2C: 2 wires, multiple slaves, 400kHz. SPI: 4 wires, fast, short range. UART: async, 2 wires, simple. CAN: automotive, noise-resistant. RS-485: long distance.

RTOS
Deterministic scheduling: hard (absolute) vs soft (flexible) real-time. FreeRTOS: task, queue, semaphore, timer. Zephyr: modern, IoT-focused. Preemptive scheduler.

Interrupt & Timer
Interrupt: external event interrupts CPU, ISR runs. Timer interrupt: periodic task. Interrupt vs polling: CPU efficiency. Interrupt priority management (NVIC).

Power Management
Light Sleep, Deep Sleep, Hibernate modes. Wakeup sources: timer, GPIO, UART. ESP32 deep sleep ~10uA. Battery life = capacity_mAh / average_current_mA.

IoT Architecture
Sensor -> Edge (MCU processing) -> Fog (local server) -> Cloud (storage/analytics) -> UI layers. Edge computing: reduce latency, save bandwidth.

MQTT
Lightweight pub/sub protocol. Broker (Mosquitto, HiveMQ) distributes messages. Topic: home/livingroom/temperature. QoS 0/1/2. Always use TLS encryption.

OTA & Security
OTA: update firmware without physical access. Dual-bank: download while running. Rollback on failure. Firmware signing, secure boot, OWASP IoT Top 10.

Embedded Linux
Yocto/Buildroot for minimal image. Cross-compilation: compile ARM code on x86. Device tree: hardware description. uBoot bootloader. systemd vs BusyBox init.

26. NUMERICAL METHODS & SCIENTIFIC COMPUTING

First number representation and error types, then root/system solving, then interpolation, then integration, then optimization, then statistical methods.

IEEE 754 Floating Point
sign(1b)+exponent(8/11b)+mantissa(23/52b). Machine epsilon ~2^-52 for double. Inf, NaN special values. Catastrophic cancellation: subtracting close numbers.

Numerical Error Types
Truncation error: from mathematical approximation. Round-off error: from FP representation. Absolute error: |x-xa|. Relative error: |x-xa|/|x|. Condition number.

Bisection Method
Root in [a,b]: test midpoint, halve interval. Guaranteed convergence but slow (linear). Requires f(a)*f(b)<0. Reliable starting point for other methods.

Newton-Raphson
x_{n+1}=x_n-f(x_n)/f'(x_n). Quadratic convergence: very fast near root. Sensitive to starting point, divergence risk. Effective when close to solution.

Linear System Solving
Gaussian elimination O(n3). LU decomposition: reusable. QR: numerically stable. Iterative (Jacobi, Gauss-Seidel): for large sparse systems. scipy.linalg.

Interpolation
Lagrange: N+1 points -> degree-N polynomial. Spline: piecewise smooth. Cubic spline: image resampling. Runge's phenomenon: oscillation with high-degree poly.

Numerical Integration
Trapezoid rule: linear approximation. Simpson 1/3: quadratic, more accurate. Gaussian quadrature: optimal node selection. scipy.integrate.quad. Error O(h^n).

ODE Solvers
Euler: simple, O(h) error. Runge-Kutta 4: standard, O(h4). Adaptive step: control error by adjusting step size. scipy.integrate.solve_ivp interface.

Eigenvalue Computation
Power iteration: finds largest eigenvalue. QR algorithm: finds all eigenvalues. scipy.linalg.eigh for symmetric. PageRank, PCA, spectral clustering.

Monte Carlo Methods
Numerical estimation via random sampling. Pi estimation: classic example. MCMC: sample from high-dimensional distributions. Bayesian inference, simulation.

FFT Applications
Signal filtering: remove frequencies. Large integer/polynomial multiplication O(n log n). Convolution theorem: spatial conv = frequency multiply. scipy.fft.

Numerical Optimization
Gradient-free: Nelder-Mead, genetic algorithm. First-order: gradient descent, SGD. Second-order: Newton, BFGS, L-BFGS (memory efficient). scipy.optimize.

26 Chapters, 350+ Terms Complete!

This guide covers all critical CS curriculum topics ordered by learning sequence. Mathematics -> Algorithms -> OS -> Networks -> DB -> Security -> ML/AI -> DevOps -> ... Each concept builds on the previous. Good luck, Alican!

>> Follow the order: don't skip math, algorithms will be much easier

>> Match each term to a project: theoretical knowledge gains meaning in practice

>> Use Linux commands daily: the terminal will make you faster

>> Write tests, profile, monitor: working code is not enough

>> Practice system design: the focus of big tech interviews